

Exp 3

Program 1 - One Client , One Server

// server.py

```
import socket
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(("127.0.0.1", 8080))
server.listen(1)
print("Server waiting for connection...")
client_socket, addr = server.accept()
message = client_socket.recv(1024).decode()
characters = len(message)
words = len(message.split())
sentences = message.count('.')
result = f"""
characters = {characters}
words = {words}
sentences = {sentences}
"""
client_socket.send(result.encode())
client_socket.close()
server.close()
```

// client.py

```
import socket
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(("127.0.0.1", 8080))
message = input("Enter paragraph: ")
client.send(message.encode())
result = client.recv(1024).decode()
print("\nResult from Server: ")
print(result)
client.close()
```

Program 2 - Multiple Client, One Server

// server.py

```
import socket
import threading

def handle_client(client_socket):
    message = client_socket.recv(1024).decode()
    characters = len(message)
    words = len(message.split())
    sentences = message.count('.')
    result = f"""
    characters = {characters}
    words = {words}
    sentences = {sentences}
    """
    client_socket.send(result.encode())
    client_socket.close()

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(("127.0.0.1", 8080))
server.listen(5)
print("Concurrent Server Running...")

while True:
    client_socket, addr = server.accept()
    print(f"Connected to {addr}")
    thread = threading.Thread(
        target = handle_client,
        args = {client_socket,}
    )
    thread.start()
```

// Execute

python/python3 filename.py

Program 3 - Group Chat

// server.py

```
import socket
import threading

clients = {}

def handle_client(client_socket, name):
    while True:
        try:
            message = client_socket.recv(1024).decode()
            if not message:
                break
            print(f"{name}: {message}")

            if message.startswith("@all"):
                msg = f"{name} : {message[5:]}"
                for client in clients.values():
                    client.send(msg.encode())

            elif message.startswith("@"):
                parts = message.split(" ", 1)
                if len(parts) < 2:
                    continue
                target_name = parts[0][1:]
                msg = parts[1]

                if target_name in clients:
                    msg = f"{name} (private) : {message}"
                    clients[target_name].send(msg.encode())

        except:
            break

    del clients[name]
    client_socket.close()

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(("127.0.0.1", 8080))
server.listen()
print("server is listening...")
```

```

while True:
    client_socket, addr = server.accept()
    name = client_socket.recv(1024).decode()
    clients[name] = client_socket
    print(f"{name} connected from {addr}")
    thread = threading.Thread(
        target = handle_client,
        args = (client_socket, name)
    )
    thread.start()

```

// client.py

```

import socket
import threading

def receive__msg():
    while True:
        try:
            message = client.recv(1024).decode()
            print("\n" + message)
        except:
            break

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(("127.0.0.1", 8080))
name = input("Enter your name: ")
client.send(name.encode())
thread = threading.Thread(target = receive__msg)
thread.start()
while True:
    msg = input()
    client.send(msg.encode())

```

Exp 6

Install

1. `sudo apt update`
2. `sudo apt install nmap -y`
3. `nmap --version`

Program 1:

`hostname -I, ip addr`

1. Active Device

`nmap -sn 10.10.72.137`

2. Tcp syn scan

`sudo nmap -sS 10.10.72.137`

3. Service version detection

`nmap -sV 10.10.72.137`

4. Os detection

`sudo nmap -O 10.10.72.137`

5. Script scanning

`nmap --script http-title 10.10.72.137`

Program 2:

1. Scan a given network range and identify all active hosts

`nmap -sn 10.10.72.0/24`

2. Identify the top 5 most commonly open ports on a specific target

`nmap --top-ports 5 10.10.72.137`

3. Determine the MAC address of a target device using NMAP.

`nmap -sn 10.10.72.137`

4. Perform a scan to detect the presence of HTTP and HTTPS services on a target network.

`nmap -p 80,443 10.10.72.137`

5. Find out if a particular host has FTP service running on it.

`nmap -p 21 10.10.72.137`

6. Identify the SSH version running on a given host.

```
nmap -sV -p 22 10.10.72.137
```

7. Scan a range of IP addresses and list all hosts that have Telnet service running.

```
nmap -p 23 10.10.72.0/24
```

8. Determine the operating system of a target host using NMAP.

```
sudo nmap -O 10.10.72.137
```

9. Identify any SQL services running on a given network.

```
nmap -p 3306 10.10.72.137
```

10. Find out if a specific host has Remote Desktop Protocol (RDP) enabled.

```
nmap -p 3389 10.10.72.137
```

11. Scan a target network and determine if any hosts are running DNS services.

```
nmap -p 53 10.10.72.0/24
```

12. Detect if a host has SNMP (Simple Network Management Protocol) enabled.

```
nmap -p 161 10.10.72.137
```

13. Perform a scan to identify any SMTP (Simple Mail Transfer Protocol) servers on a network.

```
nmap -p 25 10.10.72.0/24
```

14. Determine if a target network has any active FTP servers allowing anonymous login.

```
nmap --script ftp-anon -p 21 10.10.72.137
```

15. Find out if any hosts in a network are running vulnerable versions of the Apache HTTP server.

```
nmap --script http-vuln* -p 80 10.10.72.137
```

16. Detect if a target host has any open NFS (Network File System) shares.

```
nmap -p 2049 10.10.72.137
```

17. Identify the presence of any MySQL database servers on a given network

```
nmap -p 3306 10.10.72.0/24
```

18. Scan a network to determine if any hosts have the Remote Procedure Call (RPC) service running.

```
nmap -p 111 10.10.72.137
```

19. Detect if a specific host has any open VNC (Virtual Network Computing) ports.

```
nmap -p 5900 10.10.72.137
```

20. Perform a scan to identify any hosts with the Secure Shell (SSH) service running on non-default ports.

```
nmap -p -sV 10.10.72.137 | grep ssh
```

EXP-7

1. sudo apt update
2. sudo apt install scapy -y
3. sudo scapy

Program 1: ICMP

```
packet = IP(dst="8.8.8.8")/ICMP()
```

Program 2: UDP

```
packet = IP(dst="8.8.8.8")/UDP(dport=53)/"Hello UDP"
```

Program 3: DNS

```
packet =  
IP(dst="8.8.8.8")/UDP(dport=53)/DNS(rd=1,qd=DNSQR(qname="google.com"))
```

Program 4: TCP

```
packet = IP(dst="example.com")/TCP(dport=80)/Raw(load="GET / HTTP/1.1\r\nHost  
:example.com\r\n\r\n")
```

Program 5: TTL

```
for ttl in range(1, 6):  
    packet = IP(dst="8.8.8.8", ttl=ttl)/UDP(dport=33434)  
    reply = sr1(packet, timeout = 2, verbose=0)  
    If reply:  
        print(reply.src)
```

EXP-9

Network	IPv4	Subnet	Gateway
PC0	192.168.1.62	255.255.255.224	192.168.1.33
PC1	192.168.1.126	255.255.255.224	192.168.1.97
RO FE 0/0	192.168.1.33	255.255.255.224	
RO Serial 0/0/0	192.168.1.65	255.255.255.224	
R1 FE 0/1	192.168.1.97	255.255.255.224	
R1 Serial 0/0/0	192.168.1.66	255.255.255.224	
R)->CLI	192.168.1.96	255.255.255.224	192.168.1.66
R1->CLI	192.168.1.32	255.255.255.224	192.168.1.65